

작은 모델이 지휘한다

참조 구현과 효과 검증은 다르다

라우터 · A/B 사다리 · 의도 이력 · 출력 예산

동작하는 배선을 검증한다 . 품질 동등 절감 효과는 별도 A/B 가 필요하다 .

숫자마다 증거의 종류를 붙인다

측정 ≠ 가정 ≠ 인과 효과

API 저장 로그 : 80 건 언어 비교 + 30 건 사고 예산 스왑

새 로컬 실측 : 대응 번역문 1,012 쌍의 토큰나이저 인코딩

브라우저 · mock : 코드 동작 검증 / 비용 시나리오 : 정책 효과 실측 아님

이번 정정의 새 유료 API 호출은 0 회다 .

네 가지 원칙은 그대로 쓴다

필요한 결과만 , 적합한 설정으로

- ① 영어는 토큰 효율과 추론 정확도에 유리할 수 있다 . 사용자에게는 필요한 언어로 답한다 .
- ② KISS/DRY/YAGNI: 불필요한 코드 · 전체 재출력 · 재작업을 줄인다 .
- ③ 설명 길이와 사고 예산을 과제에 맞춘다 . 정확도 게이트를 함께 둔다 .
- ④ 스펙에 결정을 먼저 적는다 . 스펙 작성 비용과 실제 재작업 변화를 함께 잔다 .

라우팅은 단어보다 작업 목적을 본다

파일 업로드 '구현' → IMPLEMENT

설계 · 구현 · 설명 목적을 파일 · 검색 · 변환 같은 부수 단어보다 먼저 본다 .

구성한 반례 테스트는 회귀 검증이지 실사용 오류율 추정이 아니다 .

기본 tier 단가는 모델의 공식 가격이 아니라 명시적 비용 시나리오다 .

A 와 B 는 다른 실패를 처리한다

내부 AC 실패 / 사용자 반려

A: 테스트가 검출한 실패를 턴 안에서 처리 .

B: 사용자 반려를 의도 이력과 함께 판정 .

검출할 AC 가 부실하면 A 가 놓친다 . 문자열 검사만으로 기능을 보증하지 않는다 .

새 작업은 별도의 상태를 가진다

task_id 별 IntentTracker

이전 작업의 anchor·streak·rung 을 새 작업에 전달하지 않는다 .

다른 작업을 번갈아 실행해도 각자의 의도와 시도 횟수를 유지한다 .

초기화 전 inner 호출은 명시적 오류 , respec 은 실행 중단으로 처리 .

입력 증가분은 정의부터 통일한다

4,700 을 더한다 . 7,200 이 아니다 .

turn_growth_tok 은 직전 출력 · 사용자 발화 등을 포함한 총 입력 증가분 .

출력 2,500 을 다시 더하던 이중 계상을 제거 .

4,700 은 페르소나 시나리오를 맞춘 값이다 . 독립 실측 캘리브레이션은 아니다 .

상위 모델 진단 뒤 적용도 비용이다

\$0.05875 / 같은 단계 · 같은 가정

입력 1,800, file 범위 1.6, reset 600 → 입력 예산 3,480.

LARGE 진단 400 출력 + MID 적용 2,500 출력을 함께 계상 .

tier 단가와 고정 입력 예산의 시나리오다 . 실제 전달문 비용은 usage 로 측정한다 .

정액제 웹 경로도 공짜는 아니다

증분 API 비용과 총비용을 구분

EXTERNAL=0 은 추가 종량 API 금액만 세는 모델링 선택 .

구독료 · 사람의 스펙 작성 시간 · 복사 / 검토 비용은 별도다 .

라우터가 웹 세션의 품질 · 약관 적합성을 자동 보증하지 않는다 .

사다리 재생은 정책 A/B 실측이 아니다

리플레이는 유용하다 . 명칭을 바꾼다 .

기존 3.06 배는 고정 발화를 다른 정책으로 처리한다는 비용 재생이었다 .

지금은 respec 의 무료 중단과 실행 비용을 분리하고 , 총 입력 증가분을 한 번만 더한다 .

A· 오케스트레이터는 범위 · 리셋 · 출력 상한 · 진단 후 적용 비용을 공통 함수로 계산 .

새 task_id 의 의도 이력을 분리해 앞 작업의 재작업 상태가 새 작업으로 새지 않게 한다 .

검증기가 실패를 놓치지 않게 한다

거짓 통과를 반례로 검사

슬라이드 240 → 999 변조를 거부 : 실제 PPTX 의 모든 텍스트와 생성 근거를 대조 .

모바일 1 열 → 2 열 변조 , 동작 없는 메모 HTML 을 브라우저 AC 가 거부 .

실패 assertion 은 exit 1. shell pipefail 로 파이프 뒤에서도 실패를 보존 .

mock 은 요청 구조 · 계산 검증일 뿐 새 live 측정으로 소개하지 않는다 .

API usage 를 다룰 때의 세 가지 규칙

캐시를 두 번 빼지 않는다

Gemini promptTokenCount 는 캐시 입력을 포함 . cost() 에 전체 prompt 와 cached 를 전달 .

Gemini Content 는 role + parts. 모델 응답의 thoughtSignature 와 functionCall id 를 보존 .

툴 내용은 두 번째 요청에서 처음 입력되고 세 번째 요청에서 재전송된다 .

요청 실패 · 절단 · 키 없음은 미검증 / 실패로 기록 . 3 번더 ‘확인됨’을 미리 출력하지 않는다 .

캐시 적격성은 모델별로 다르다

Haiku 4.5: 최소 4,096 토큰

Sonnet 4.6: 1,024 / Opus 4.6: 4,096 (공식 문서 확인일 2026-09-20)

짧은 프리픽스를 1K 공통 규칙으로 할인하지 않는다 . 미확인 기준은 ‘미확인’으로 둔다 .

적격성 충족은 히트 보장이 아니다 . 실제 cache usage·TTL·쓰기 / 저장료를 본다 .

캐싱의 손익은 조건으로 재현한다

기본 시나리오 62.35% 절감

고정 20,000 + 가변 1,000, 출력 1,500, 100 회, Sonnet: \$8.550 → \$3.219

히트 0, 매번 쓰기 가정은 \$10.050. 캐시 없음보다 비쌀 수 있다.

‘캐싱 90%’는 캐시 읽기 단가 할인이다. 전체 세션 절감률과 다르다.

20.8 배는 페르소나 시나리오로 보존

기존 20.8 배 / 수정 기본 19.0 배

기존 : A 6 턴 / B 2 턴 , 출력 확대 2.9 배 , effort 0.35, 한국어 사고 승수 1.44 등 가정 .

기본 : 사고 언어 승수는 1.0, 최소 길이 미달 캐싱은 미적용 .

두 조건 모두 일부 실파일 인코딩이 포함된 비용 모형 . 실제 품질 동등 A/B 가 아니다 .

`python demo/compare_personas.py --scenario legacy` 로 기존 발표 수치 재현 .

SDD 절감은 '턴 감소' 가정에 달린다

기존 71.0% / cold-cache 68.7%

40 턴 → 12 턴은 모델의 입력 가정 . 스펙이 실제 이만큼 줄었다는 관측은 없다 .

스펙 작성 : 입력 3,000 + 출력 4,000 비용을 이미 포함한다 .

기존 약 71% 는 캐시 예열 비용 제외 . 기본은 사이클별 첫 쓰기까지 포함한다 .

주장 유지 : 짧고 결정적인 스펙을 우선 시도하되 재작업 · 품질 · 총비용으로 검증한다 .

숫자를 직접 확인할 수 있게 남긴다

원자료 · 산식 · 기준일 · 재현 명령

results/audit_metrics.json: 저장 로그 통계 + 로컬 측정 + 시나리오를 분리한 지표 .

docs/CORRECTIONS.md: 무엇을 유지하고 무엇을 바꿨는지 상세 설명 .

SOURCES.md: 공식 단가 , 논문 표 , 외부 케이스의 비교 조건 .

원본 110 행은 보존 . 없는 원자료를 재측정 없이 만들어 넣지 않는다 .

작게 바꾸고 , 자기 usage 와 품질로 확인한다

영어 · 간결함 · 스펙 · 컨텍스트 관리

주장은 유지한다 : 불필요한 생성 · 사고 · 재전송은 줄일 가치가 있다 .

배수는 모델 · 단가 · 과제 · 시나리오에 붙여 말한다 . 절감만으로 품질을 대신하지 않는다 .

정확도 · 재작업 · 총비용을 함께 측정해 실제 배포 정책을 선택한다 .